How Fragile Is Your Watermark? Training-Free Structural Removal of Neural Audio Watermarks

APSIPA ASC 2026 · Likhith Kumara, IITM

Paper ID: 1571317245

Chair: Jiantao Zhou (MSF)

12 NOVEMBER 2026

https://arxiv.org/pdf/2608.16566

https://github.com/i-618/audio-watermark-fragility

Outline

- Introduction and motivation
- Related works
- Methodology
- Experimental Setup
- Results and discussion
- Conclusion and future work

Introduction and motivation

- Audio watermark model should embed watermark with high imperceptibility (Audio Quality - PESQ) and robustness (Accuracy - %).

- Audio Watermark benchmarks usually report robustness as a table of bit accuracies for a fixed set of distortions.

- This does not give info on where the audio watermark is present and how easy it is to remove it.

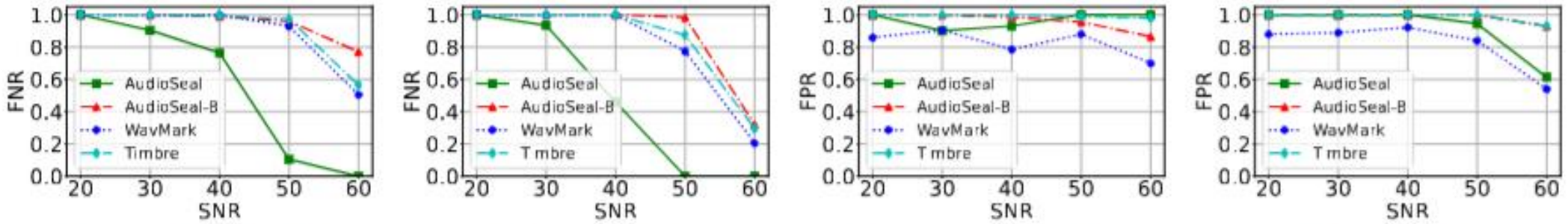
Related works

Raw Bench, Sony

Table 5: *Comparison of bitwise (top) and full-message (bottom) robustness for the considered models under various attacks (columns, see abbreviations in Table 2). For each model, we evaluate the strict (S) and loose (L) settings (Eval column). An asterisk (*) indicates that the model has been retrained with the strict attacks, and a checkmark (✓) indicates an accuracy of 0.99 or above.*

Model	Eval	GN	BN	RV	DC	DE	LM	LP	HP	EQ	TS	TJ	PI	GA	QN	PS	EN	DA	MP	OG	AA
AS	S	✓	✓	.87	✓	✓	.98	✓	.96	.91	.97	✓	.18	✓	✓	.62	.96	.52	✓	✓	✓
SC	S	.63	.98	.80	.96	.91	.83	✓	.93	.92	.41	.86	✓	✓	✓	.63	.33	.32	.54	.61	.93
TI	S	.98	✓	.96	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	.64	.60	✓	✓	✓
WM	S	.83	✓	.89	.98	.95	.95	✓	✓	✓	.82	.98	✓	.98	✓	✓	.00	.00	.42	.89	✓

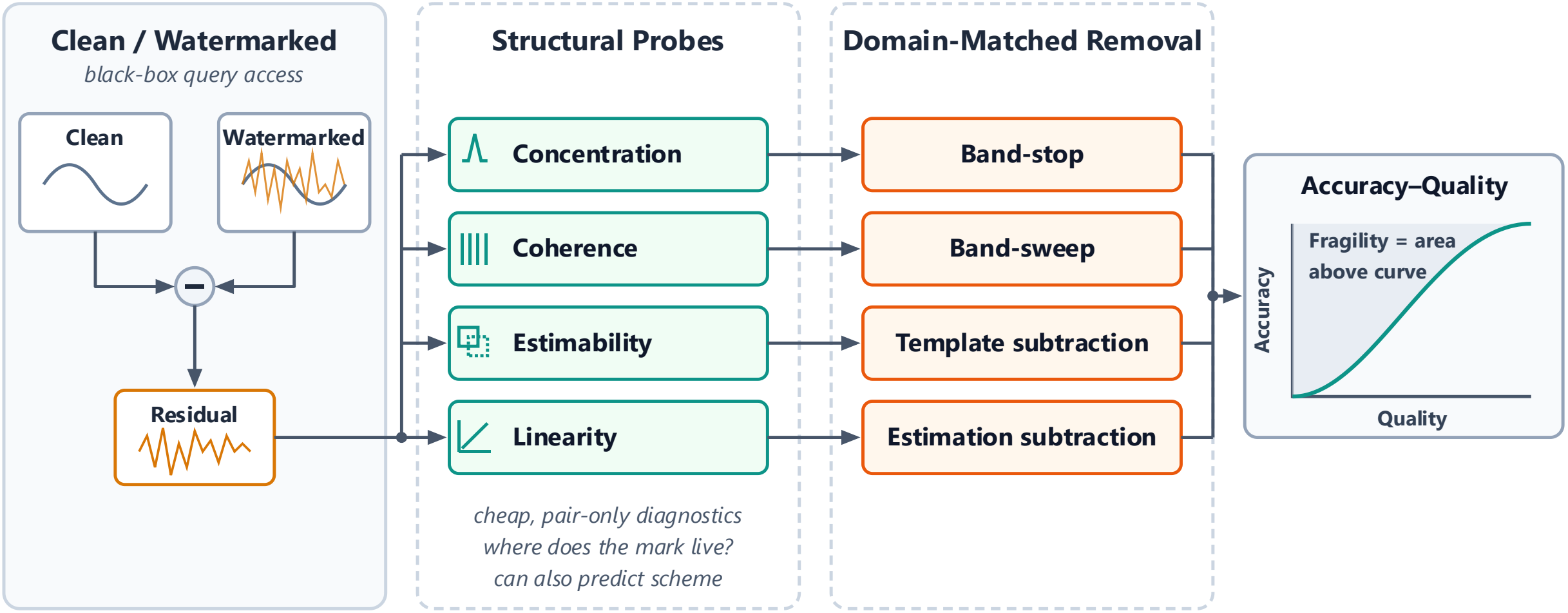
AudioMark Bench, Google



(a) FNR AudioMarkData (b) FNR LibriSpeech (c) FPR AudioMarkData (d) FPR LibriSpeech

Figure 7: Detection results under white-box watermark-removal and watermark-forgery perturbations.

Methodology



Experimental Setup

Watermark models

1. **audiowmark**
2. **WavMark**
3. **AudioSeal**
4. **SilentCipher**
5. **DNN-AWM**
6. **Timbre**
7. **WMCodec**
8. **VoiceMark**
9. **AWARE**
10. **AlignMark**

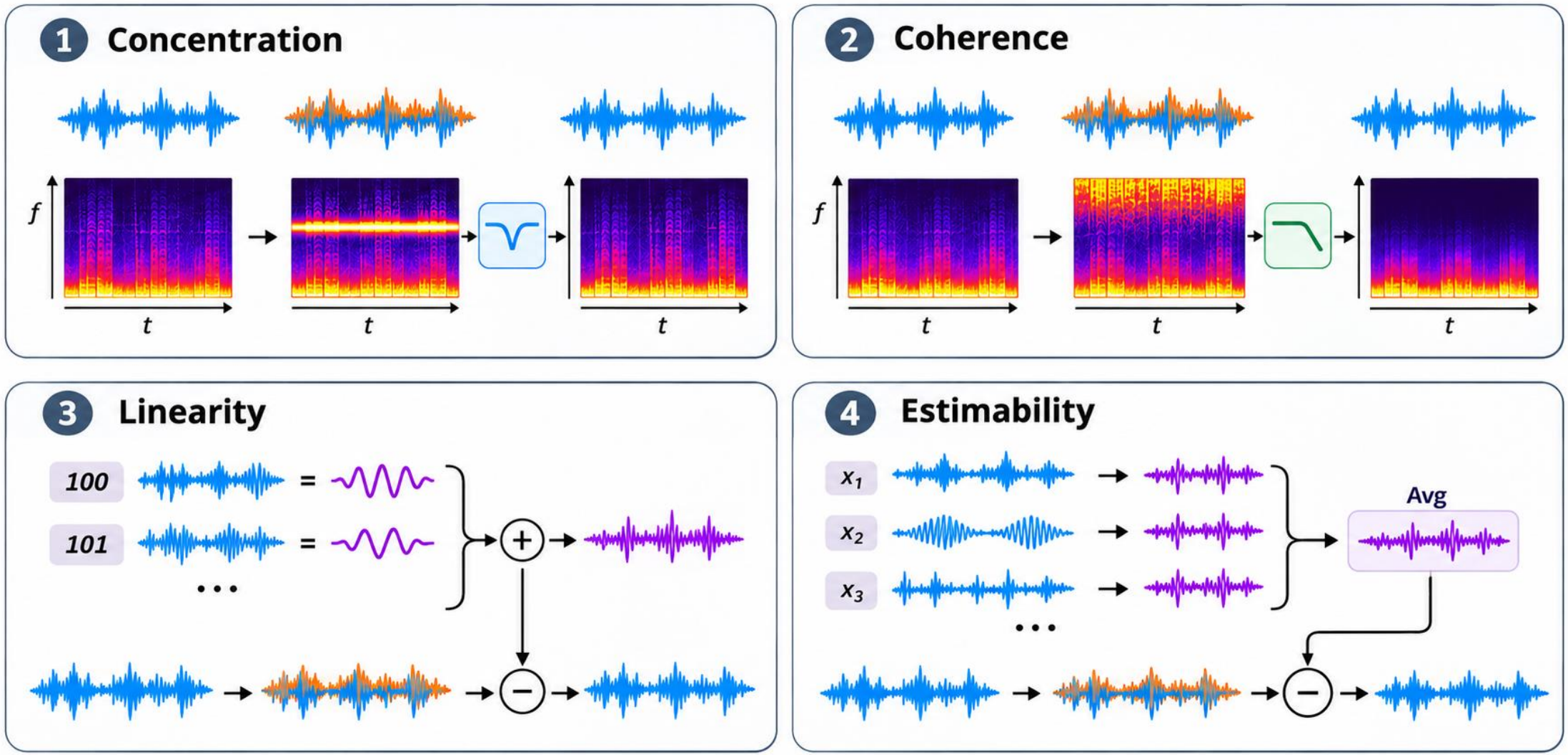
Dataset

- Speech – **LibriSpeech**
- Multi-Speaker – **VCTK**
- Music - **MUSDB18**
- Singing – **VocalSet**
- Synthetic-Speech - (**XTTS-v2** + **F5-TTS** + **CosyVoice 2**)

Probes and attacks

- **Concentration** - band-stop notch - mark energy in one narrow band
- **Coherence** - band-sweep(low-pass) - payload replicated across bands
- **Estimability** - template subtraction - host-independent template
- **Linearity** - estimation subtraction - additive in the payload bits

The four probes and subsequent removal



Results of attacks

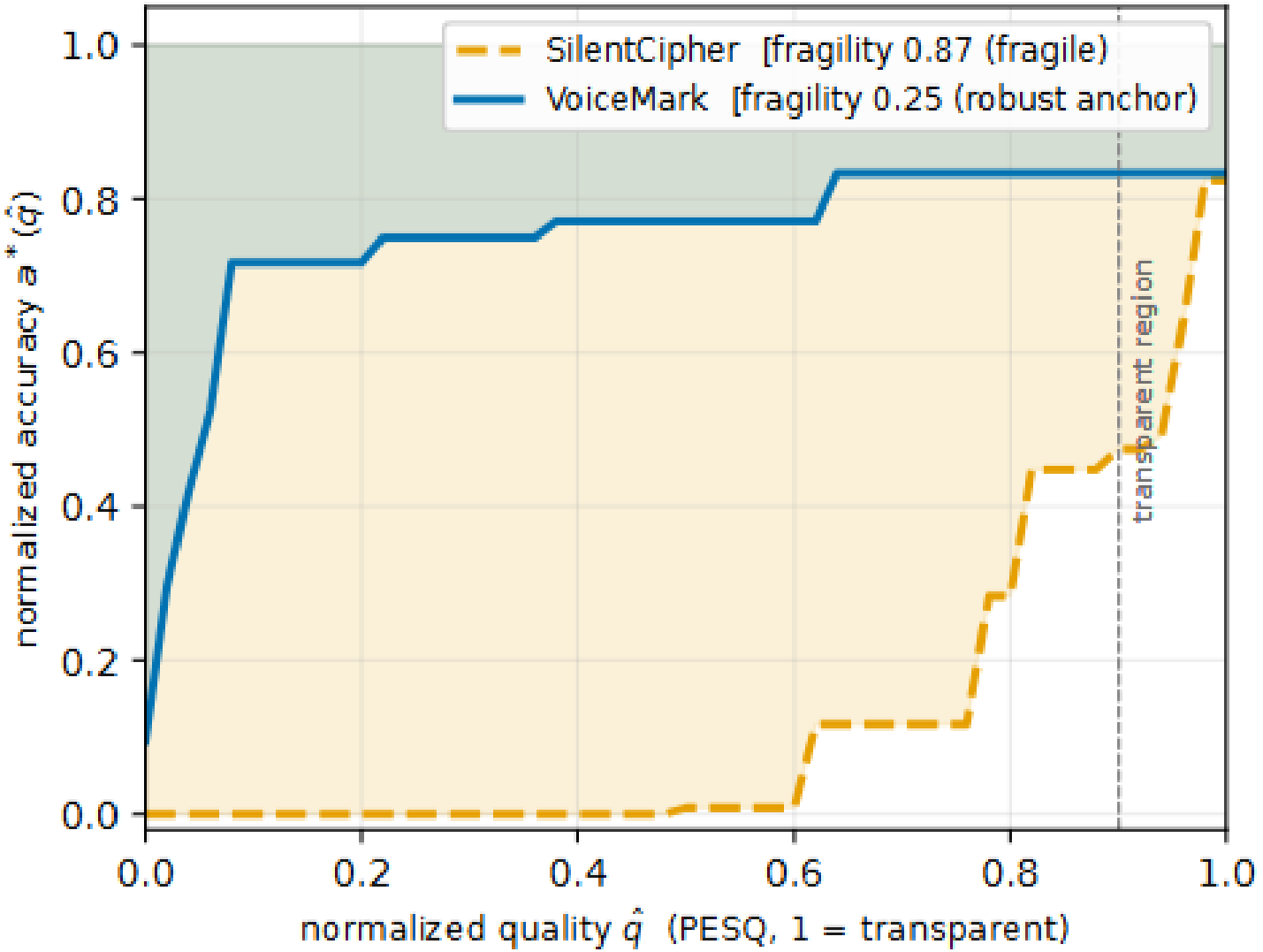
TABLE II

PROBE-MATCHED REMOVALS. LOWER BIT-ACC = PAYLOAD REMOVED (CHANCE $\approx 50\%$); DET = DETECTION FLAG THERE. MEANS $\pm 95\%$ CI, N=100 CLIPS (89 FOR AUDIOWMARK; 84 FOR ITS FRAGILITY). **PESQ IS SPEECH CONTENT ONLY** (N=60); AUDIO-MODE VISQOL COVERS ALL FIVE CONTENT TYPES.

Scheme	matched attack	bit-acc	det	PESQ	ViSQOL	frag[a]
Removed at near-transparent quality:						
audiowmark	template subtr.	$\mathbf{48.4 \pm 2.9}$	**0.00**	4.22	4.62	0.85 ± 0.03
WavMark	template subtr.	$\mathbf{5.9 \pm 4.6}$	**0.06**	4.17	4.47	0.87 ± 0.05
AudioSeal	band-stop	60.7 ± 2.8	**0.05**	3.62	4.55	0.80 ± 0.03
SilentCipher	RMS-adaptive subtr.	$\mathbf{10.3 \pm 4.1}$	0.20	3.84	4.22	0.84 ± 0.03
DNN-AWM[b]	band-sweep	58.9 ± 3.6	1.00	3.34	3.36	0.62 ± 0.04
Timbre	estimation subtr.	62.6 ± 1.2	1.00	3.99	4.30	0.74 ± 0.04
Robust anchors (mark only dents at audible quality):						
WMCodec	template subtr.	57.4 ± 2.1	1.00	1.47	3.27	0.27 ± 0.04
VoiceMark	template subtr.	61.9 ± 2.6	0.54	1.52	3.50	0.28 ± 0.05
AWARE	template subtr.	81.0 ± 2.5	0.09	2.39	4.28	0.38 ± 0.05
AlignMark	template subtr.	61.3 ± 2.6	1.00	1.69	3.77	0.39 ± 0.06

[a] Threshold-free fragility, (5); one attack family per scheme.
[b] Borderline: removed, but just under the > 3.5 transparency line.

Threshold free fragility metric



Some Insights

- Models like Alignmark, VoiceMark, Aware and WMCodec have lower fragility since they embed watermark adjusting to content.
- DNN-AWM and Timbre lie on the border with watermark removed while the audio quality is affected, the other four models are fragile.
- Using the Probe values got from clean-watermark audio pair the watermarking model can be identified with over 83% accuracy.

Conclusion and future work

- Current method gives a low compute technique to diagnose and remove the watermarks.
- Once the weakness of the watermark is recognized, the architecture can be modified to design a more robust model.
- Synchronization attack is excluded in the current work due to limitation of objective quality of audio after the attack.

Thank You — Questions?

Please reach out at da24s026@smail.iitm.ac.in

Appendix

Concentration – notch removal

- Subtract clean audio from watermarked audio to isolate the mark.
- Take its spectrum and find the band holding the most energy.
- If one band holds most of it, notch that band out.

Linearity - template subtraction

- Embed messages that differ by one bit and compare the outputs.
- Fit each bit's change as a separate additive piece.
- If the pieces add up cleanly, subtract their sum from the clip.

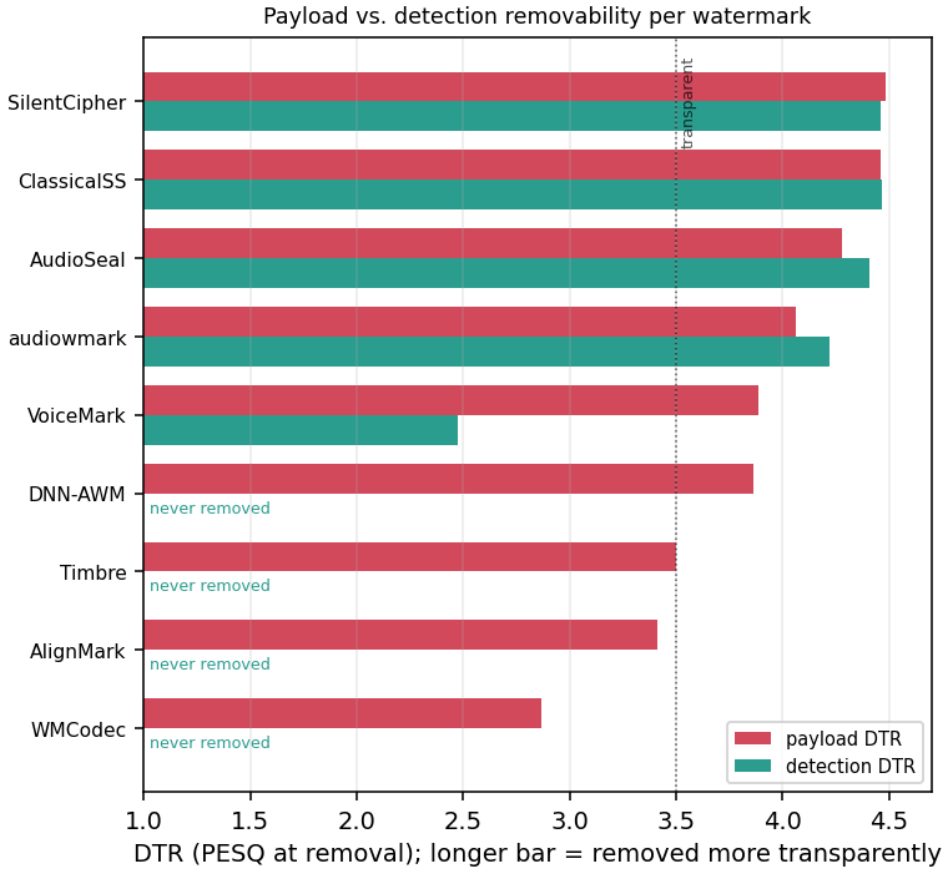
Coherence - low pass filter

- Low-pass the watermarked clip at a high cutoff and decode it.
- Lower the cutoff step by step, decoding each time.
- Stop at the highest cutoff where bit accuracy falls to chance.

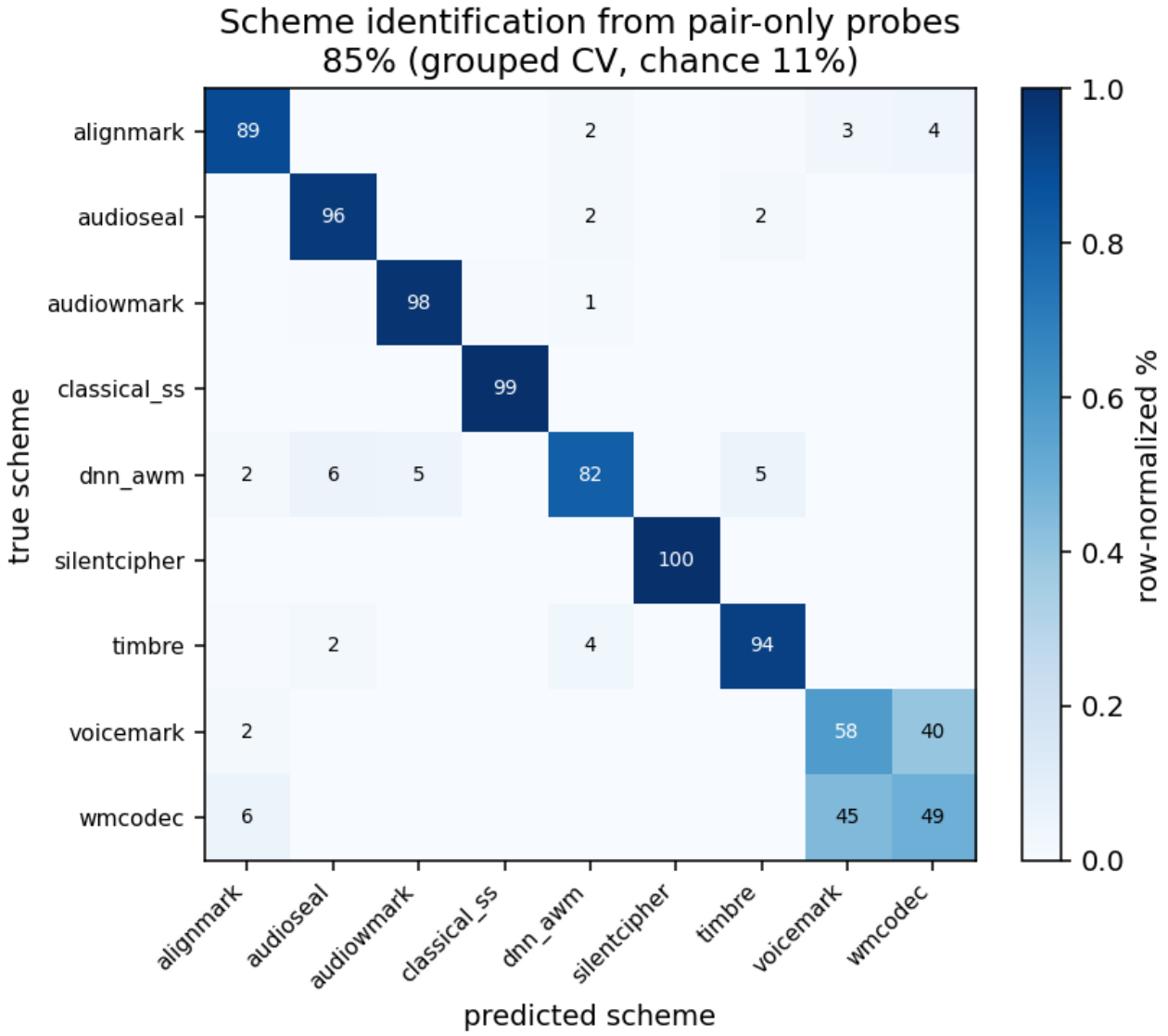
Estimability - estimation subtraction

- Compute the mark (watermarked minus clean) on several different clips.
- Check how similar those marks are to each other.
- If they match, average them into one template and subtract it.

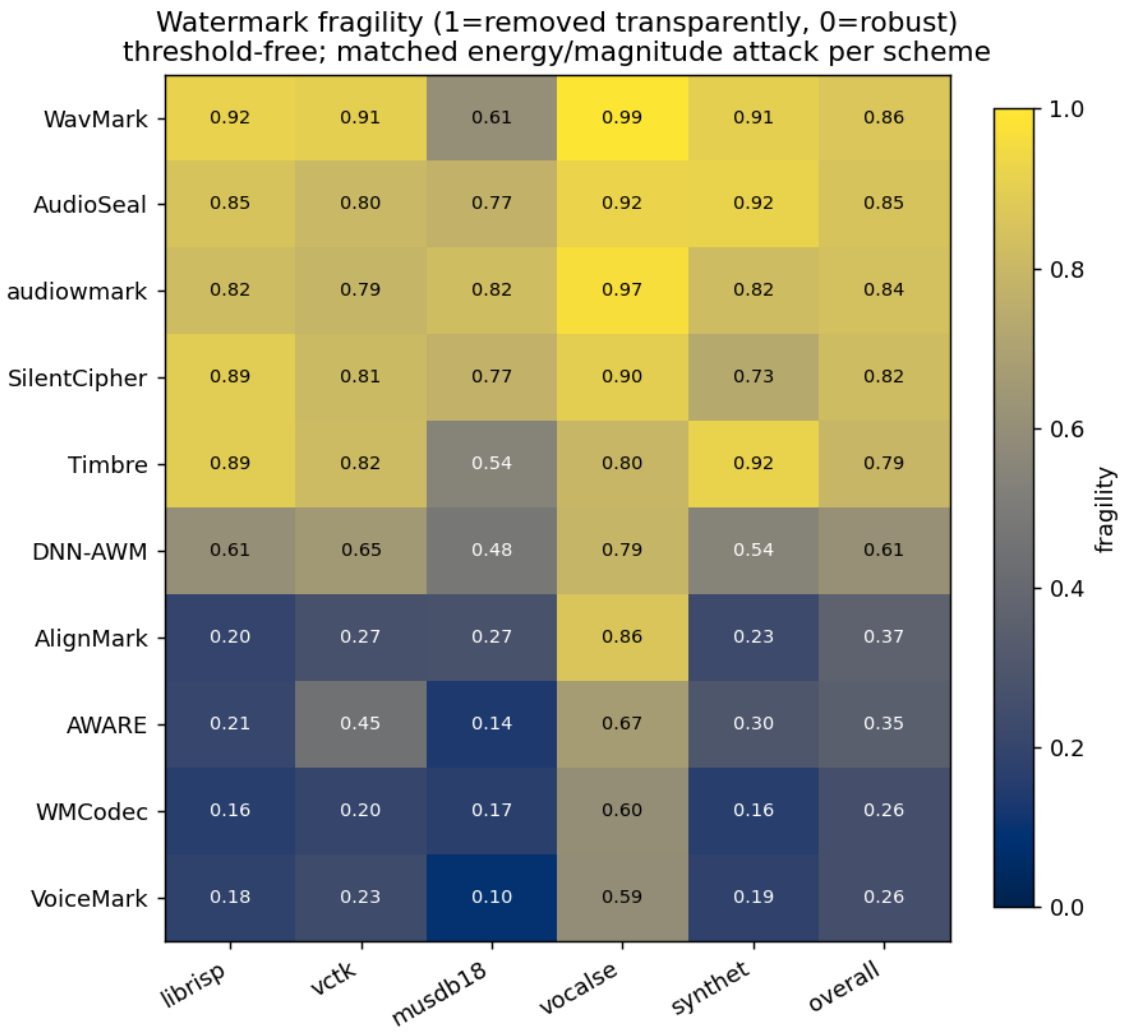
Comparing payload and detection

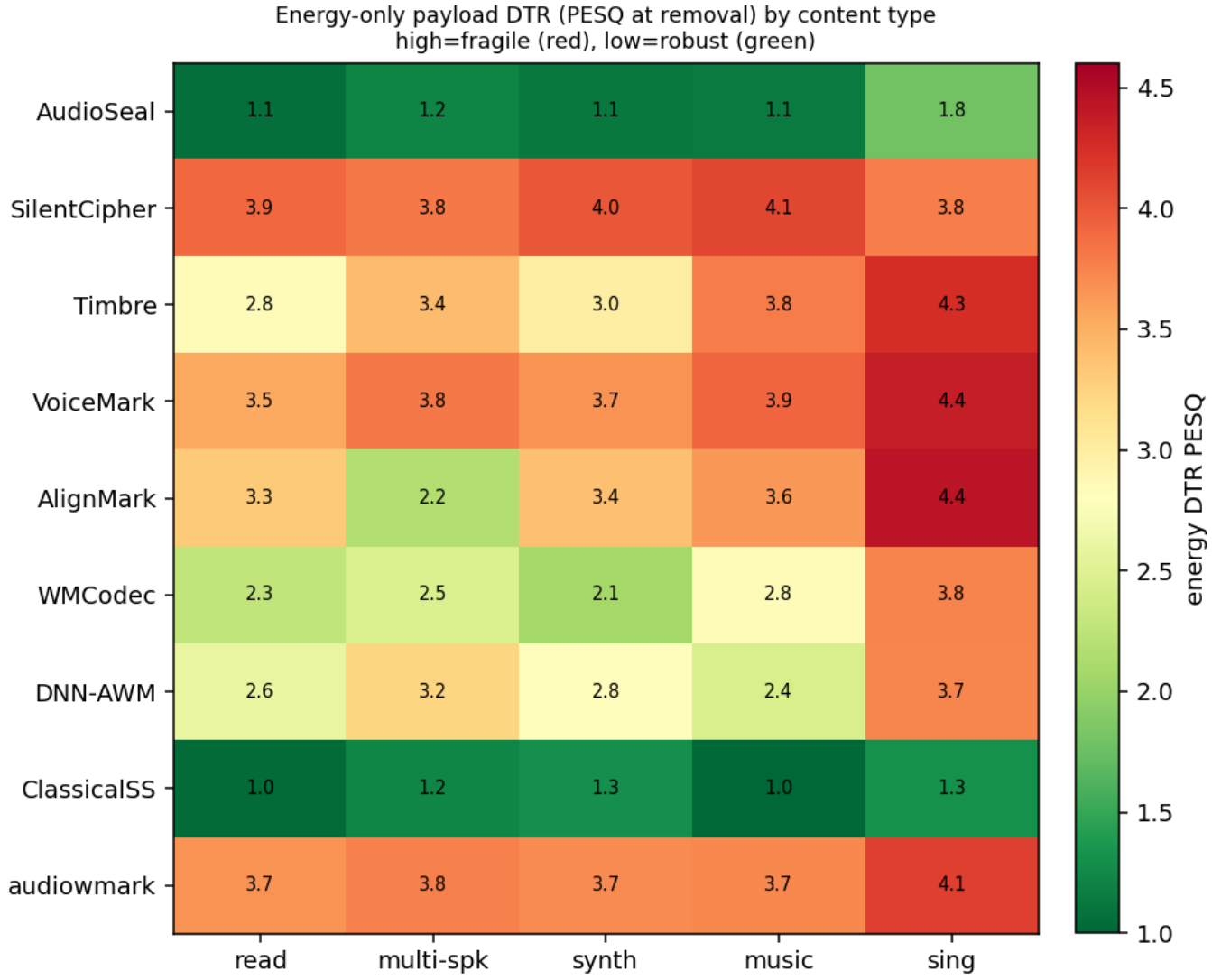


Scheme id from probe values

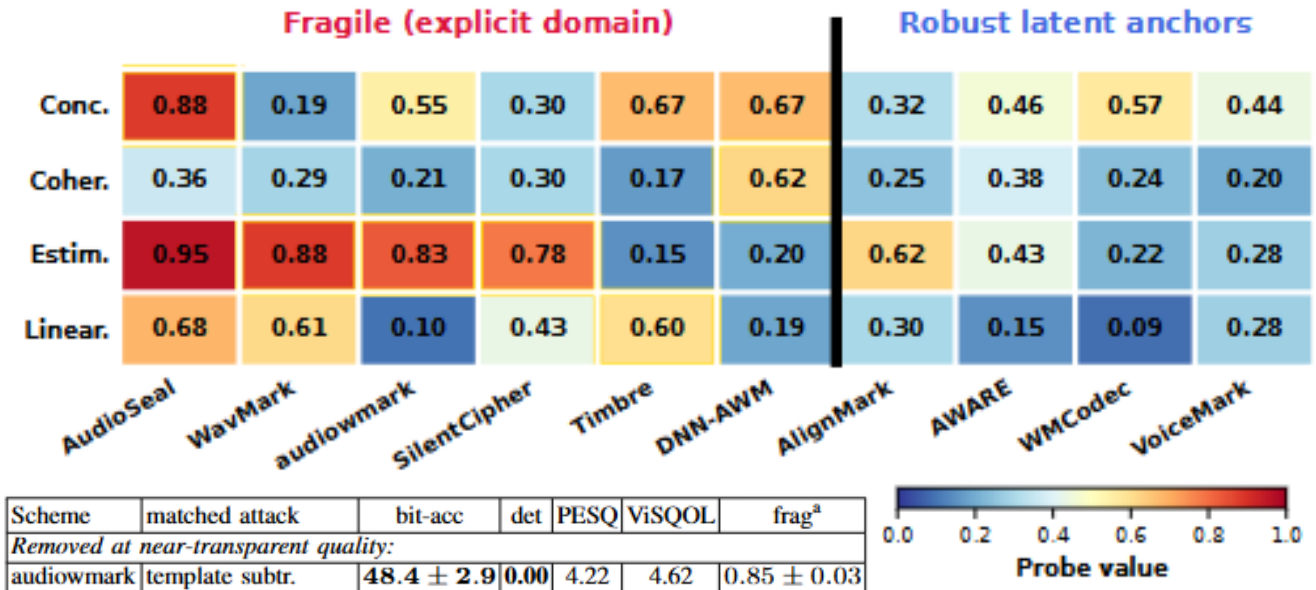


Content based comparison



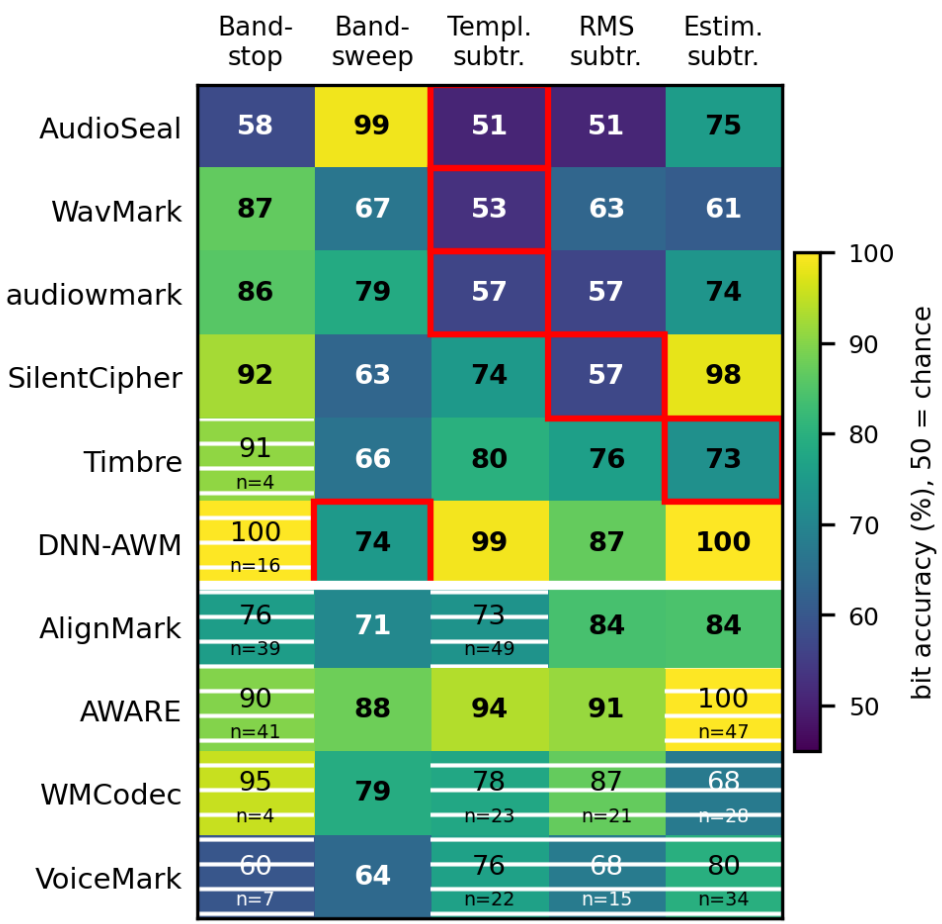


All Probes and All attacks



Scheme	matched attack	bit-acc	det	PESQ	ViSQOL	frag[a]
Removed at near-transparent quality:						
audiowmark	template subtr.	**48.4 ± 2.9**	**0.00**	4.22	4.62	0.85 ± 0.03
WavMark	template subtr.	**5.9 ± 4.6**	**0.06**	4.17	4.47	0.87 ± 0.05
AudioSeal	band-stop	60.7 ± 2.8	**0.05**	3.62	4.55	0.80 ± 0.03
SilentCipher	RMS-adaptive subtr.	**10.3 ± 4.1**	0.20	3.84	4.22	0.84 ± 0.03
DNN-AWM[b]	band-sweep	58.9 ± 3.6	1.00	3.34	3.36	0.62 ± 0.04
Timbre	estimation subtr.	62.6 ± 1.2	1.00	3.99	4.30	0.74 ± 0.04
Robust anchors (mark only dents at audible quality):						
WMCodec	template subtr.	57.4 ± 2.1	1.00	1.47	3.27	0.27 ± 0.04
VoiceMark	template subtr.	61.9 ± 2.6	0.54	1.52	3.50	0.28 ± 0.05
AWARE	template subtr.	81.0 ± 2.5	0.09	2.39	4.28	0.38 ± 0.05
AlignMark	template subtr.	61.3 ± 2.6	1.00	1.69	3.77	0.39 ± 0.06

[a] Threshold-free fragility, (5); one attack family per scheme.
[b] Borderline: removed, but just under the > 3.5 transparency line.



```
def concentration_probe(host, watermarked):
    """Fraction of residual energy contained in its strongest spectral sixteenth."""
    P = np.abs(np.fft.rfft(watermarked - host)) ** 2; bw = len(P) // 16
    return max(P[i:i+bw].sum() for i in range(0, len(P)-bw, bw)) / (P.sum() + 1e-8)

def coherence_probe(residual_spectra):
    """Cross-band alignment of message-dependent residual subspaces."""
    D = residual_spectra - residual_spectra.mean(0, keepdims=True)
    return np.linalg.svd(D, compute_uv=False)[:2].dot(np.linalg.svd(D,
compute_uv=False)[:2]) / (D**2).sum()

def estimability_probe(footprints, host_envelope):
    """Similarity of residual footprints after removing their host-envelope
component."""
    R = footprints - np.outer(footprints @ host_envelope, host_envelope)
    R /= np.linalg.norm(R, axis=1, keepdims=True) + 1e-8; return np.mean((R @
R.T)[np.triu_indices(len(R), 1)])

def linearity_probe(delta_a, delta_b):
    """Context-invariance of the perturbation induced by a payload-bit flip."""
    return abs(np.dot(delta_a, delta_b))
    / (np.linalg.norm(delta_a) * np.linalg.norm(delta_b) + 1e-8)
```

```python
def band_sweep_attack(y, cutoff_hz, sr):
    """Remove all STFT bins above a frequency cutoff, preserving lower-band content."""
    S = librosa.stft(y); S[librosa.fft_frequencies(sr=sr) > cutoff_hz, :] = 0
    return librosa.istft(S, length=len(y))

def band_stop_attack(y, center_hz, bandwidth_hz, sr):
    """Remove a targeted frequency band while retaining the original STFT phase."""
    S = librosa.stft(y); f = librosa.fft_frequencies(sr=sr); S[abs(f-center_hz) <=
bandwidth_hz/2, :] = 0
    return librosa.istft(S, length=len(y))

def template_subtraction_attack(y, template, alpha):
    """Subtract a fixed learned watermark magnitude template from every STFT frame."""
    S = librosa.stft(y); A = np.maximum(abs(S) - alpha * template[:, None], 0)
    return librosa.istft(A * np.exp(1j * np.angle(S)), length=len(y))

def rms_adaptive_attack(y, template, alpha):
    """Subtract a watermark template scaled to each frame's signal RMS."""
    S = librosa.stft(y); rms = np.sqrt((abs(S)**2).mean(0) + 1e-8)
    return librosa.istft(np.maximum(abs(S)-alpha*template[:,None]*rms,
0)*np.exp(1j*np.angle(S)), length=len(y))

def estimation_subtraction_attack(y, carrier, alpha):
    """Subtract an estimated message-independent watermark carrier from STFT magnitudes."""
    S = librosa.stft(y); A = np.maximum(abs(S) - alpha * carrier[:, None], 0)
    return librosa.istft(A * np.exp(1j * np.angle(S)), length=len(y))
```

PROBES 1–2, ILLUSTRATIVE

Concentration & coherence, in code

concentration probe

```
diff = stft(wm) - stft(clean)
energy = abs(diff) ** 2
band_share = energy[band] / energy.sum()
if band_share > threshold:
    peak_band = argmax(energy)
```

coherence probe

```
for cutoff in sweep_grid:
    acc = decode(lowpass(wm, cutoff)).bit_acc
    if acc < chance_floor:
        best_cutoff = cutoff
        break
```

→ notch attack

```
def notch_attack(audio, peak_band, sr):
    b, a = iirnotch(peak_band, Q=30, fs=sr)
    return lfilter(b, a, audio)
```

→ low-pass attack

```
def lowpass_attack(audio, cutoff, sr):
    sos = butter(8, cutoff, fs=sr,
output="sos")
    return sosfiltfilt(sos, audio)
```

Illustrative sketches following the paper's method — not literal source lines.

PROBES 3–4, ILLUSTRATIVE

Estimability & linearity, in code

estimability probe

```
diffs = [wm_i - clean_i for i in train_pairs]
template = mean(diffs)
corr = correlation(diffs, template)
if corr > threshold:
    estimable = True
```

linearity probe

```
per_bit = [estimate_contribution(bit)
           for bit in payload_bits]
residual = additive_fit_error(per_bit)
if residual < threshold:
    linear = True
```

→ template subtraction attack

```
def template_attack(audio, template):
    return audio - template
```

→ estimation subtraction attack

```
def estimation_attack(audio, per_bit):
    return audio - sum(per_bit)
```

Illustrative sketches following the paper's method — not literal source lines.